

GIT Version Control System [GVCS]

Git was created by Linus Torvalds for the development of the Linux Kernel

What is Git?

Git is a version control system.

This means that you can "capture" the exact state of your files and can come back to it any time if you feel like you made some mistakes while changing something there.

When we as Developers are working on a project, then we continuously keep making changes to the code according to the project requirements.

But then sometime we may need to go back to check the previous versions of the code too.

Basic concepts

Your local code

This is the work that you do on your computer. Any edits, formats, features or development work that you have in your computer is your local code.

Staged site

Once you are happy with the changes or amount of work done, you can mark it as ready for stage. It means that you are declaring that these lines of code are ready to be committed.

The server

Once you are feel ready with the files you have staged, you can send them out to the server which stores all your code so that other people can use it too. Now your files can be viewed by other people and be worked on.

We may do this to see if:

• The previous version worked better.

• To see what changes were done and when.

• And In Identifying bugs.

• Git Helps us track and manage all these changes.

What is GitHub?

GitHub is a platform which allows you to store this version history on the cloud.

How are Git and GitHub different?

.....

• Git - is a tool that helps us track & manage all the changes that were done over time.
• Git is run and maintained on your Local system.

Whereas,

• Github - is a website where you host your project in the form of Git repositories
• Github is completely Cloud-based.

+++++

Basics:

.....

Install:

.....

```
# apt install git -y
# yum install git -y
```

Check version:

.....

```
# git --version
```

When you start using Git, you'll need to put in a username and an email id :

```
# git config --global user.name "username"
# git config --global user.email "username@email.com"
# git config --global --list
OR
# git config --global -l
```

Use the --unset option to remove a setting:

.....

```
# git config --unset --global user.email
```

```
# git clone https://github.com/account_name/repo.git
# cd repo/
# ls -a
.git
```

TO initializing a git repository in the current directory (specified in your terminal)

.....

```
# mkdir dir
# cd dir
# git init
```

```
# ls -a
.git/
```

```
# git add .
OR
# git add FILENAME
```

Look at just the repository specific settings

```
.....  
# cat .git/config  
[core]  
    repositoryformatversion = 0  
    filemode = true  
    bare = false  
    logallrefupdates = true  
[remote "var_name"]  
    url = https://github.com/sinhakiara/rce.git  
    fetch = +refs/heads/*:refs/remotes/var_name/*
```

Create a branch:

```
.....  
# git checkout -b first  
-b: branch name
```

[Used to switch and create a new branch]

Switch to branch

```
.....  
# git checkout first
```

List branch:

```
.....  
# git branch
```

List Remote Branches

```
.....  
# git branch -a
```

```
--  
# cat >> readme.md  
llll  
--  
# git diff  
diff --git a/readme.md b/readme.md  
index 039727e..11d14ae 100644  
--- a/readme.md  
+++ b/readme.md  
@@ -1,2 @@  
  lol  
+llll  
.....  
# git add readme.md
```

OR

```
# git add .
```

```
# git commit -m "lol1"
[first 835e51c] lol1
1 file changed, 1 insertion(+)
```

```
Merge 'test' branch with 'main' branch:
.....
```

```
# git checkout main
# git merge test
```

```
To View Your Commits
.....
# git log
```

```
show-branch
.....
# git show-branch
.....
```

```
commit to remote git repository
.....
```

```
1. Setup Name and Email
.....
```

```
# git config --global user.name "Your Name"
# git config --global user.email "your_email@whatever.com"
```

```
# mkdir hello
# cd hello
```

```
# git clone URL
# cat > hello.py
```

```
#!/usr/bin/python3
print("Hello, World")
```

```
2. Check the status of the repository
.....
```

Use the git status command to check the current status of the repository.

```
# git status
```

3. Add Changes

.....

```
# git add hello.py
# git status
# git commit -m "Changes hello"
```

Getting a listing of what changes have been made is the function of the git log command.

```
# git log
```

One Line Histories

.....

```
# git log --pretty=oneline
```

Controlling Which Entries are Displayed

.....

```
# git log --pretty=oneline --max-count=2
# git log --pretty=oneline --since='5 minutes ago'
# git log --pretty=oneline --until='5 minutes ago'
# git log --pretty=oneline --author=<your name>
# git log --pretty=oneline --all
```

Public Repo Workflow

.....

1. Create a public/private repo on GitHub:

<https://github.com/sinhakiara/DevOps-Demo>

2. Clone the repo:

```
# git clone https://github.com/sinhakiara/DevOps-Demo.git
```

3. Create a branch:

```
# git checkout "test"
# git branch
```

4. Make changes in 'test' branch:

```
# cat >> README.md
```

LOL

```
# cat > index.html
<h1>ulala</h1>
```

```
# git add .
```

```
# git status
```

```
# git commit -m "Home web page in test branch"
```

5. Merge 'test' branch in 'main' branch:

```
`
```

```
# git checkout main
```

```
# git merge test
```

6. Remote add the repo:

```
`
```

```
# git remote add origin https://github.com/sinhakiara/DevOps-Demo.git
```

```
# cat .git/config
```

7. Generate Token :

```
.....
```

Settings --> Developer Settings --> Personal Access Token --> Generate New Token

```
# git config --global credential.helper store
```

8. Push the data:

```
``
```

```
# git push -u origin main
```

```
username:
```

```
pass: token
```

```
.git/
```

```
-----  
username: sinhakiara
```

```
# git clone https://github.com/sinhakiara/DAl.git
```

```
-----  
  
https://github.com/Dheerajmadhukar/karma_v2
```

Git Internals:

```
-----
```

The .git directory

```
# ls -C .git
```

The Object Store

.....

```
# ls -C .git/objects
```

You should see a bunch of directories with 2 letter names. The directory names are the first two letters of the SHA1 hash of the object stored in git.

Deeper into the Object Store

.....

```
# ls -C .git/objects/<dir>
```

- Look in one of the two-letter directories.
- You should see some files with 38-character names.
- These are the files that contain the objects stored in git.
- These files are compressed and encoded, so looking at their contents directly won't be very helpful, but we will take a closer look in a bit.

Config File

.....

```
# cat .git/config
```

- This is a project-specific configuration file.

Branches and Tags

.....

```
# ls .git/refs
# ls .git/refs/heads
# ls .git/refs/tags
# cat .git/refs/tags/v1
```

Each file corresponds to a tag you created with the git tag command earlier. Its content is just the hash of the commit tied to the tag.

The HEAD File

.....

```
# cat .git/HEAD
```

The HEAD file contains a reference to the current branch.

Dumping the Latest Commit

.....

```
# git hist --all [not working now]
```

Using the SHA1 hash from the commit listed above ...

```
# git cat-file -t <hash>
# git cat-file -p <hash>
```

Finding the Tree

.....

We can dump the directory tree referenced in the commit. This should be a description of the (top level) files in our project (for that commit). Use the SHA1 hash from the "tree" line listed above.

```
# git cat-file -p <treehash>
100644 blob 28e0e9d6ea7e25f35ec64a43fe8386f90  Rakefile
040000 tree e46f374f5b36c6f02fb379044f754d795  lib
```

Dumping the lib directory

.....

```
# git cat-file -p <libhash>
```